



UNIVERSITÄT
LEIPZIG

„Breaking BMGs“

Fortgeschrittene Methoden in der Bierinformatik

SS 2026

Dr. Julia van Beesel, Max Kleineberg,
Tony Iberler, Dr. Alex Mordhorst

OBJECTIVES

1. Compare the definition of best matches and weak best matches:
 - a) Check whether every best match in (N, σ) is also a weak best match. Test this computationally, but also try to give a formal argument.
 - b) Check if a modified version of the BIC-cherry + expansion procedure yields networks that explain weak best match graphs.
2. Find a way to edit the explaining networks for (weak) best matches to become more tree like.

OBJECTIVE 1A

CHECK WHETHER EVERY BEST MATCH IS ALSO A WEAK BEST MATCH: FORMAL ARGUMENT

$$\forall x \in L(N) \wedge \tau \in \sigma(L(N)) \setminus \{\sigma(x)\}$$

$$Q(x, \tau) = \min_{\preceq} \{v \in \text{LCA}(x, y) : \sigma(y) = \tau\}$$

Weak Best Matches:

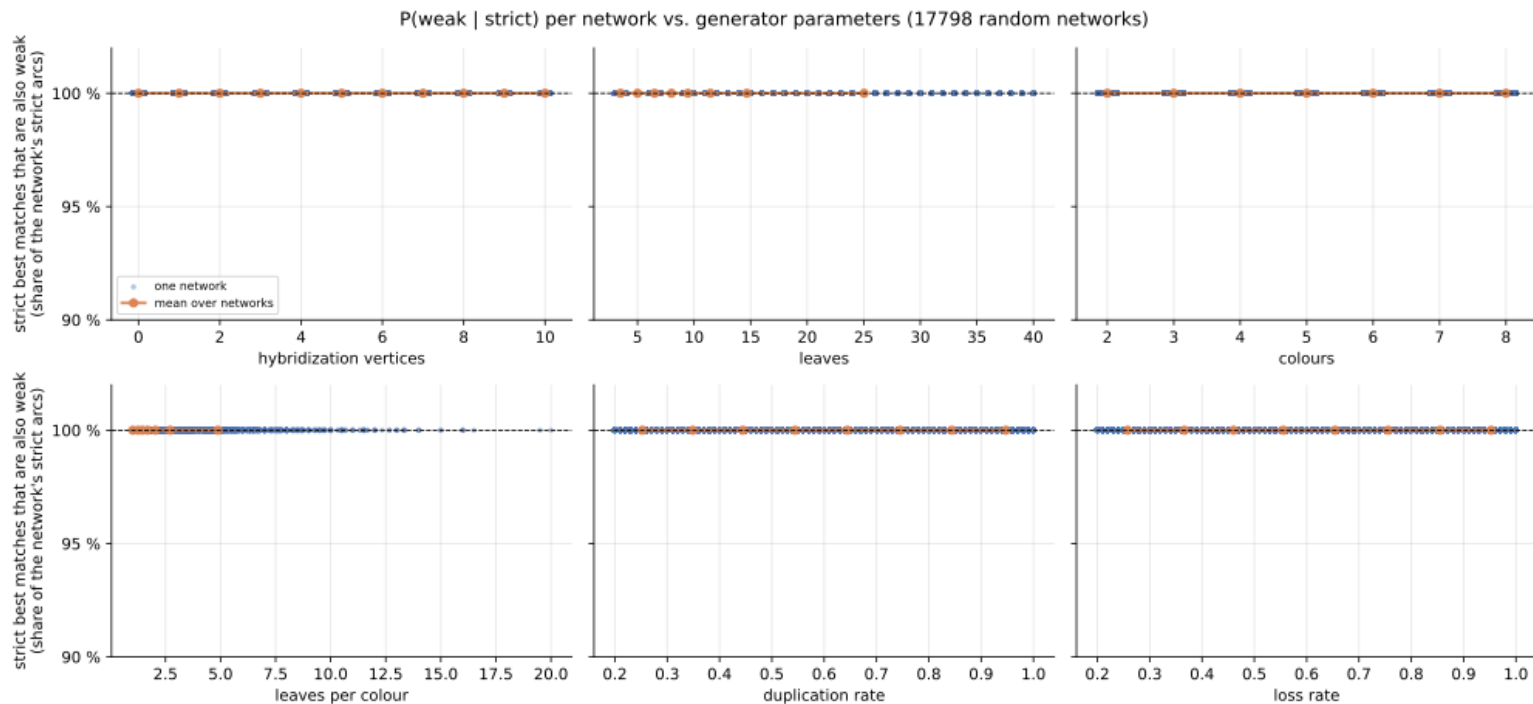
$$\forall x, y \in L(N), \sigma(y) \neq \sigma(x) : \text{LCA}(x, y) \cap Q(x, \sigma(y)) \neq \emptyset$$

Strict Best Matches:

$$\forall x, y \in L(N), \sigma(y) \neq \sigma(x) : \text{LCA}(x, y) \subseteq Q(x, \sigma(y))$$

OBJECTIVE 1A

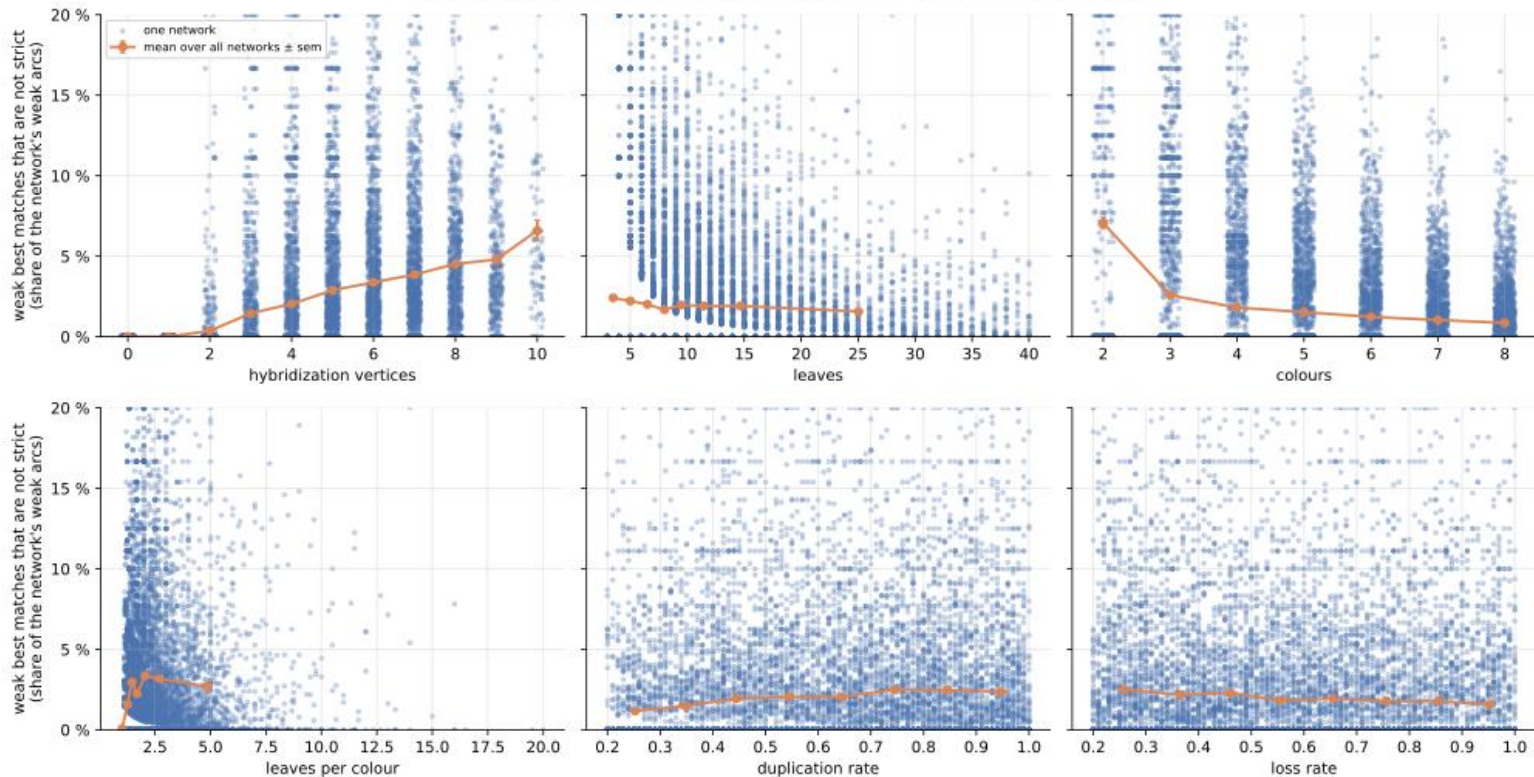
CHECK WHETHER EVERY BEST MATCH IS ALSO A WEAK BEST MATCH: COMPUTATIONAL EVALUATION



OBJECTIVE 1A

CHECK WHETHER EVERY BEST MATCH IS ALSO A WEAK BEST MATCH: COMPUTATIONAL EVALUATION

Weak-only share per network vs. generator parameters (17798 random networks)



OBJECTIVE 1B

CHECK IF A MODIFIED VERSION OF THE BIC-CHERRY + EXPANSION PROCEDURE YIELDS NETWORKS THAT EXPLAIN WEAK BEST MATCH GRAPHS.

Algorithm 2: BIC-cherry + classical expansion

Require: properly coloured digraph (G, σ) on L with the sicor-in-hub property

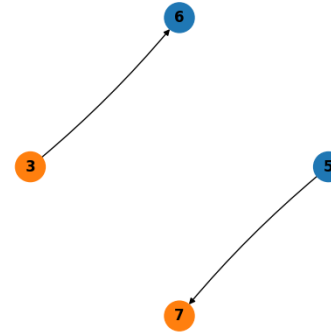
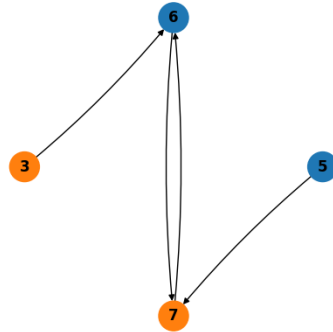
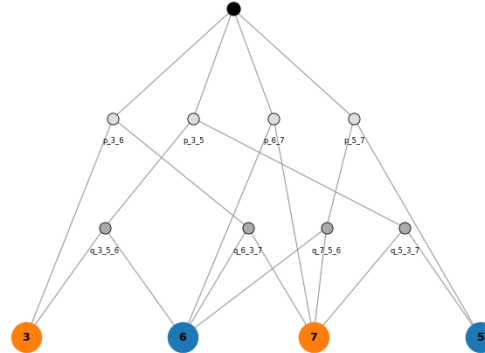
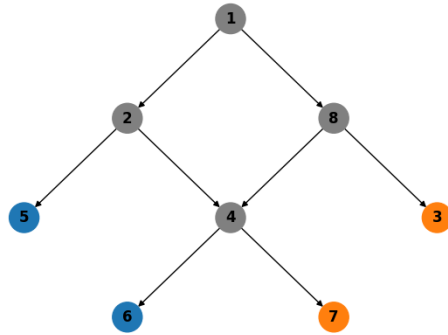
Ensure: leaf-coloured network (N, σ)

```
1:  $V(N) \leftarrow L \cup \{\rho\}$ ;  $E(N) \leftarrow \emptyset$ 
2: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                ▷ BIC-cherry network
3:   add  $p_{xy}$  with edges  $(\rho, p_{xy}), (p_{xy}, x), (p_{xy}, y)$ 
4: end for
5: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                ▷ expansion
6:   for all  $(a, b) \in ((x, y), (y, x))$  with  $(a, b) \notin E(G)$  do
7:      $b' \leftarrow \text{FINDCANDIDATE}(G, a, b)$ 
8:     add new vertex  $q$  with edges  $(p_{xy}, q), (q, a), (q, b')$                 ▷  $[ab : ab']$ 
9:   end for
10: end for
11: return  $(N, \sigma)$ 

12: function FINDCANDIDATE( $G, a, b$ )
13:    $C \leftarrow \{v \in L \setminus \{b\} : \sigma(v) = \sigma(b)\}$ 
14:   return random element of  $C$ 
15: end function
```

(OBJECTIVE „1C“)

CAVEAT: COUNTEREXAMPLE TO PROOF OF RESOLUTION VIA STRICT DEF.



OBJECTIVE 1B

CHECK IF A MODIFIED VERSION OF THE BIC-CHERRY + EXPANSION PROCEDURE YIELDS NETWORKS THAT EXPLAIN WEAK BEST MATCH GRAPHS.

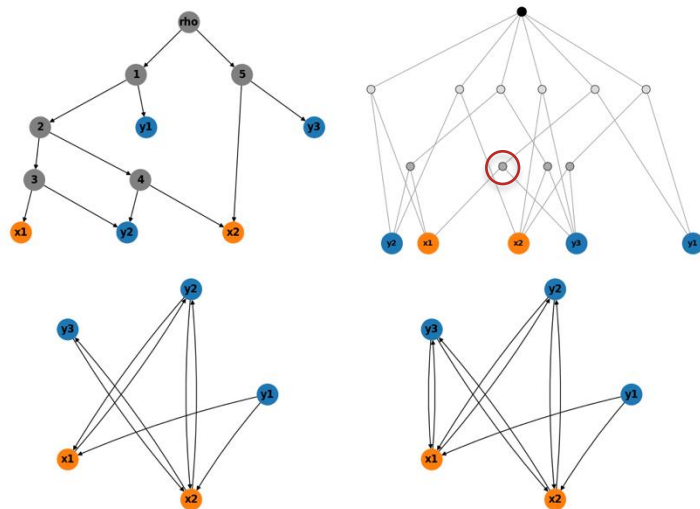
Algorithm 2: BIC-cherry + classical expansion

Require: properly coloured digraph (G, σ) on L with the sicor-in-hub property

Ensure: leaf-coloured network (N, σ)

```
1:  $V(N) \leftarrow L \cup \{\rho\}$ ;  $E(N) \leftarrow \emptyset$ 
2: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                 $\triangleright$  BIC-cherry network
3:   add  $p_{xy}$  with edges  $(\rho, p_{xy})$ ,  $(p_{xy}, x)$ ,  $(p_{xy}, y)$ 
4: end for
5: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                 $\triangleright$  expansion
6:   for all  $(a, b) \in \{(x, y), (y, x)\}$  with  $(a, b) \notin E(G)$  do
7:      $b' \leftarrow \text{FINDCANDIDATE}(G, a, b)$ 
8:     add new vertex  $q$  with edges  $(p_{xy}, q)$ ,  $(q, a)$ ,  $(q, b')$            $\triangleright [ab : ab']$ 
9:   end for
10: end for
11: return  $(N, \sigma)$ 

12: function FINDCANDIDATE( $G, a, b$ )
13:    $C \leftarrow \{v \in L \setminus \{b\} : \sigma(v) = \sigma(b)\}$ 
14:   return random element of  $C$ 
15: end function
```



OBJECTIVE 1B

CHECK IF A MODIFIED VERSION OF THE BIC-CHERRY + EXPANSION PROCEDURE YIELDS NETWORKS THAT EXPLAIN WEAK BEST MATCH GRAPHS.

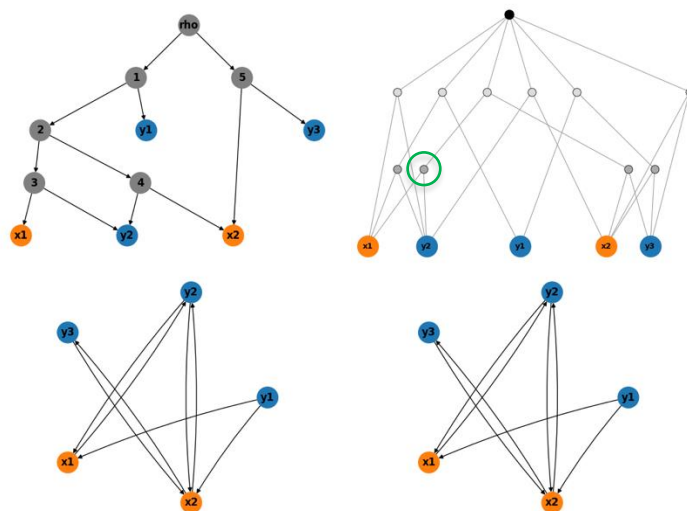
Algorithm 3: BIC-cherry + restricted expansion

Require: properly coloured digraph (G, σ) on L with the sicor-in-hub property

Ensure: leaf-coloured network (N, σ)

```
1:  $V(N) \leftarrow L \cup \{\rho\}$ ;  $E(N) \leftarrow \emptyset$ 
2: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                 $\triangleright$  BIC-cherry network
3:   add  $p_{xy}$  with edges  $(\rho, p_{xy})$ ,  $(p_{xy}, x)$ ,  $(p_{xy}, y)$ 
4: end for
5: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                 $\triangleright$  expansion
6:   for all  $(a, b) \in \{(x, y), (y, x)\}$  with  $(a, b) \notin E(G)$  do
7:      $b' \leftarrow \text{FINDCANDIDATE}(G, a, b)$ 
8:     add new vertex  $q$  with edges  $(p_{xy}, q)$ ,  $(q, a)$ ,  $(q, b')$            $\triangleright [ab : ab']$ 
9:   end for
10: end for
11: return  $(N, \sigma)$ 

12: function FINDCANDIDATE( $G, a, b$ )
13:    $C \leftarrow \{v \in L : \sigma(v) = \sigma(b), (a, v) \in E(G)\}$      $\triangleright \neq \emptyset$  if  $(G, \sigma)$  is sink-free
14:   return random element of  $C$ 
15: end function
```



OBJECTIVE 1B

CHECK IF A MODIFIED VERSION OF THE BIC-CHERRY + EXPANSION PROCEDURE YIELDS NETWORKS THAT EXPLAIN WEAK BEST MATCH GRAPHS.

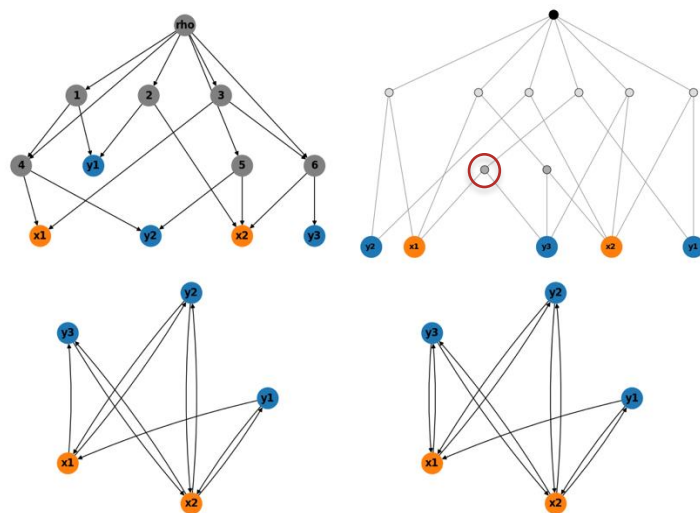
Algorithm 3: BIC-cherry + restricted expansion

Require: properly coloured digraph (G, σ) on L with the sicor-in-hub property

Ensure: leaf-coloured network (N, σ)

```
1:  $V(N) \leftarrow L \cup \{\rho\}$ ;  $E(N) \leftarrow \emptyset$ 
2: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                 $\triangleright$  BIC-cherry network
3:   add  $p_{xy}$  with edges  $(\rho, p_{xy})$ ,  $(p_{xy}, x)$ ,  $(p_{xy}, y)$ 
4: end for
5: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                 $\triangleright$  expansion
6:   for all  $(a, b) \in \{(x, y), (y, x)\}$  with  $(a, b) \notin E(G)$  do
7:      $b' \leftarrow \text{FINDCANDIDATE}(G, a, b)$ 
8:     add new vertex  $q$  with edges  $(p_{xy}, q)$ ,  $(q, a)$ ,  $(q, b')$            $\triangleright [ab : ab']$ 
9:   end for
10: end for
11: return  $(N, \sigma)$ 

12: function FINDCANDIDATE( $G, a, b$ )
13:    $C \leftarrow \{v \in L : \sigma(v) = \sigma(b), (a, v) \in E(G)\}$      $\triangleright \neq \emptyset$  if  $(G, \sigma)$  is sink-free
14:   return random element of  $C$ 
15: end function
```



OBJECTIVE 1B

CHECK IF A MODIFIED VERSION OF THE BIC-CHERRY + EXPANSION PROCEDURE YIELDS NETWORKS THAT EXPLAIN WEAK BEST MATCH GRAPHS.

Algorithm 4: BIC-cherry + expand via symmetry (weak only)

Require: properly coloured digraph (G, σ) on L with the sicor-in-hub property

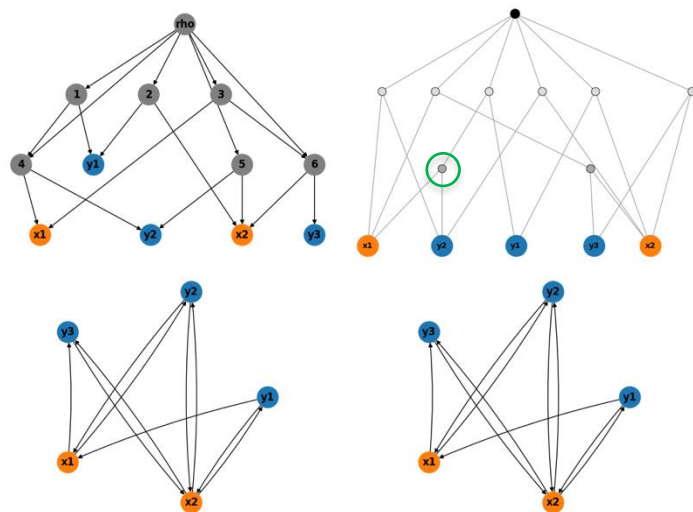
Ensure: leaf-coloured network (N, σ)

```

1:  $V(N) \leftarrow L \cup \{\rho\}$ ;  $E(N) \leftarrow \emptyset$ 
2: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                 $\triangleright$  BIC-cherry network
3:   add  $p_{xy}$  with edges  $(\rho, p_{xy})$ ,  $(p_{xy}, x)$ ,  $(p_{xy}, y)$ 
4: end for
5: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                 $\triangleright$  expansion
6:   for all  $(a, b) \in ((x, y), (y, x))$  with  $(a, b) \notin E(G)$  do
7:      $b' \leftarrow \text{FINDCANDIDATE}(G, a, b)$ 
8:     add new vertex  $q$  with edges  $(p_{xy}, q)$ ,  $(q, a)$ ,  $(q, b')$            $\triangleright [ab : ab']$ 
9:     remove edge  $(p_{xy}, a)$                                                $\triangleright$  shortcut
10:  end for
11: end for
12: return  $(N, \sigma)$ 

13: function FINDCANDIDATE( $G, a, b$ )
14:    $C \leftarrow \{v \in L : \sigma(v) = \sigma(b), (a, v) \in E(G), (v, a) \in E(G)\}$ 
15:   if  $C = \emptyset$  then  $C \leftarrow \{v \in L : \sigma(v) = \sigma(b), (a, v) \in E(G)\}$ 
16:   if  $C = \emptyset$  then  $C \leftarrow \{v \in L \setminus \{b\} : \sigma(v) = \sigma(b), (v, a) \in E(G)\}$ 
17:   if  $C = \emptyset$  then  $C \leftarrow \{v \in L \setminus \{b\} : \sigma(v) = \sigma(b)\}$ 
18:   return random element of  $C$ 
19: end function

```



OBJECTIVE 1B

CHECK IF A MODIFIED VERSION OF THE BIC-CHERRY + EXPANSION PROCEDURE YIELDS NETWORKS THAT EXPLAIN WEAK BEST MATCH GRAPHS.

Algorithm 4: BIC-cherry + expand via symmetry (weak only)

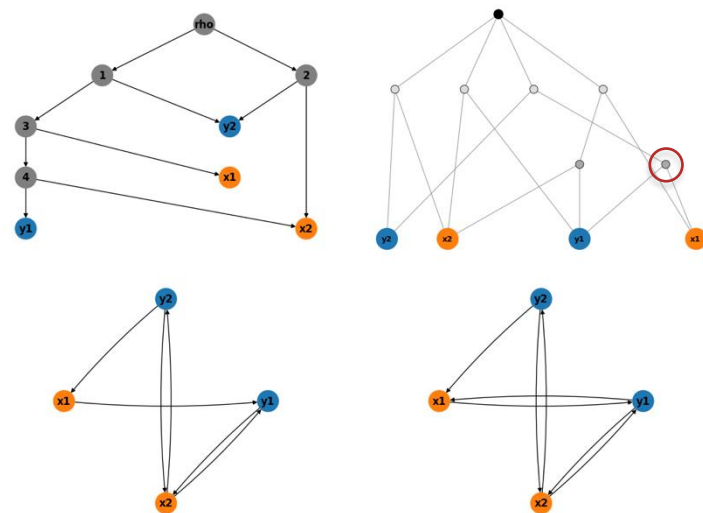
Require: properly coloured digraph (G, σ) on L with the sicor-in-hub property

Ensure: leaf-coloured network (N, σ)

```

1:  $V(N) \leftarrow L \cup \{\rho\}$ ;  $E(N) \leftarrow \emptyset$ 
2: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                 $\triangleright$  BIC-cherry network
3:   add  $p_{xy}$  with edges  $(\rho, p_{xy})$ ,  $(p_{xy}, x)$ ,  $(p_{xy}, y)$ 
4: end for
5: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do                                 $\triangleright$  expansion
6:   for all  $(a, b) \in ((x, y), (y, x))$  with  $(a, b) \notin E(G)$  do
7:      $b' \leftarrow \text{FINDCANDIDATE}(G, a, b)$ 
8:     add new vertex  $q$  with edges  $(p_{xy}, q)$ ,  $(q, a)$ ,  $(q, b')$            $\triangleright [ab : ab']$ 
9:     remove edge  $(p_{xy}, a)$                                                $\triangleright$  shortcut
10:  end for
11: end for
12: return  $(N, \sigma)$ 

13: function FINDCANDIDATE( $G, a, b$ )
14:    $C \leftarrow \{v \in L : \sigma(v) = \sigma(b), (a, v) \in E(G), (v, a) \in E(G)\}$ 
15:   if  $C = \emptyset$  then  $C \leftarrow \{v \in L : \sigma(v) = \sigma(b), (a, v) \in E(G)\}$ 
16:   if  $C = \emptyset$  then  $C \leftarrow \{v \in L \setminus \{b\} : \sigma(v) = \sigma(b), (v, a) \in E(G)\}$ 
17:   if  $C = \emptyset$  then  $C \leftarrow \{v \in L \setminus \{b\} : \sigma(v) = \sigma(b)\}$ 
18:   return random element of  $C$ 
19: end function
  
```



Algorithm 6: BIC-cherry + multilayered expansion (weak)

Require: coloured digraph (G, σ) on L , maximal number of layers K (default $K = |L|$)

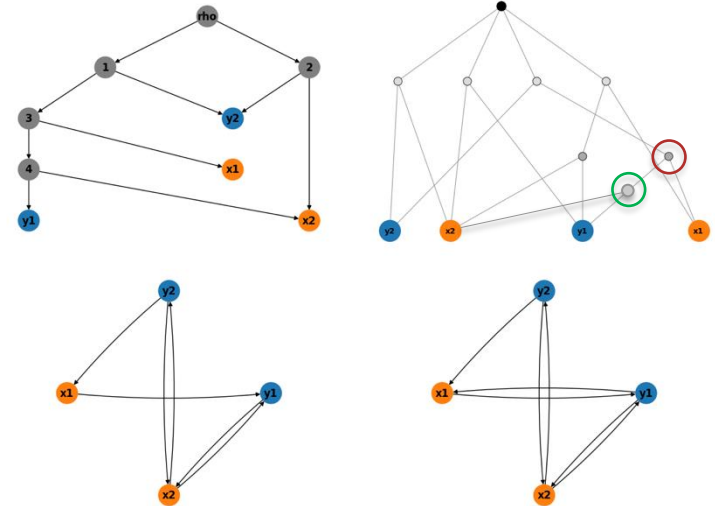
Ensure: leaf-coloured network (N, σ)

```

1:  $V(N) \leftarrow L \cup \{\rho\}$ ;  $E(N) \leftarrow \emptyset$ ;  $W \leftarrow \emptyset$ 
2: if  $|L| = 2$  then add edges  $(\rho, x)$  for all  $x \in L$  and return  $(N, \sigma)$ 
3: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do ▷ BIC-cherry network
4:   add  $p_{xy}$  with edges  $(\rho, p_{xy})$ ,  $(p_{xy}, x)$ ,  $(p_{xy}, y)$ 
5: end for
6: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do ▷ expansion, layer 1
7:   for all  $(a, b) \in ((x, y), (y, x))$  with  $(a, b) \notin E(G)$  do
8:      $(b', r) \leftarrow \text{FINDCANDIDATE}(G, a, b)$ 
9:     add new vertex  $q$  with edges  $(p_{xy}, q)$ ,  $(q, a)$ ,  $(q, b')$  ▷  $[ab : ab']$ 
10:    remove edge  $(p_{xy}, a)$  ▷ shortcut
11:    if not  $r$  then  $W \leftarrow W \cup \{(b', a, q)\}$  ▷ pending correction
12:   end for
13: end for
14:  $\ell \leftarrow 2$ 
15: while  $W \neq \emptyset$  and  $\ell \leq K$  do ▷ layers 2, 3, ...
16:    $W' \leftarrow \emptyset$ 
17:   for all  $(a, b, u) \in W$  do
18:      $(b', r) \leftarrow \text{FINDCANDIDATE}(G, a, b)$ 
19:     add new vertex  $q$  with edges  $(u, q)$ ,  $(q, a)$ ,  $(q, b')$ 
20:     remove edge  $(u, a)$ 
21:     if not  $r$  then  $W' \leftarrow W' \cup \{(b', a, q)\}$ 
22:   end for
23:    $W \leftarrow W'$ ;  $\ell \leftarrow \ell + 1$ 
24: end while
25: return  $(N, \sigma)$ 

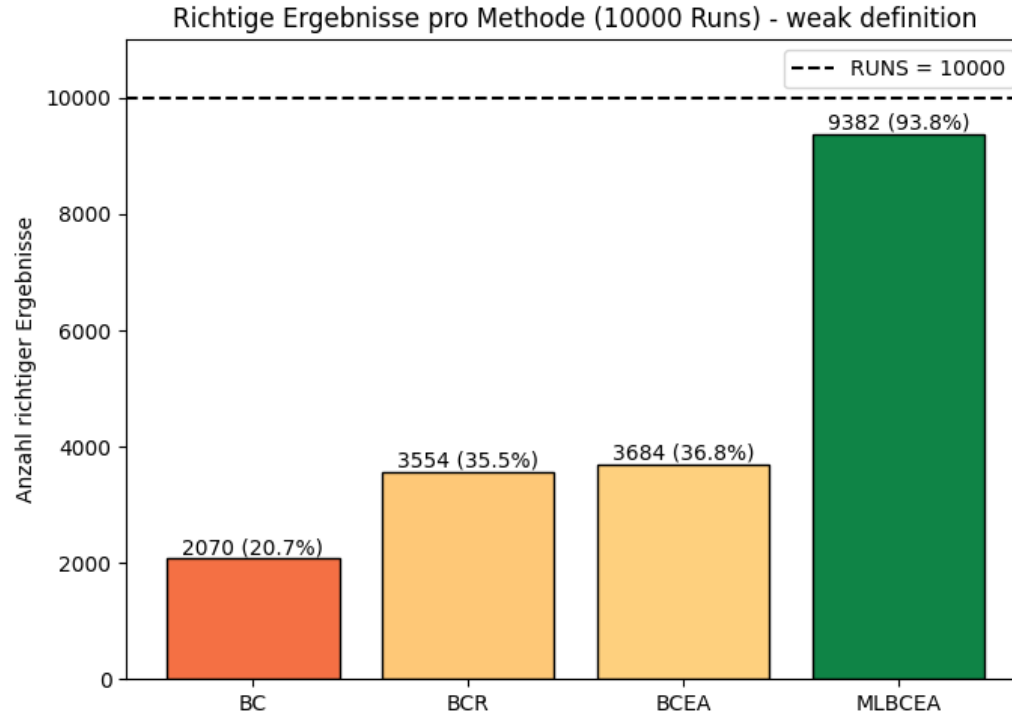
26: function FINDCANDIDATE( $G, a, b$ )
27:    $C \leftarrow \{v \in L : \sigma(v) = \sigma(b), (a, v) \in E(G), (v, a) \in E(G)\}$ 
28:   if  $C \neq \emptyset$  then return (random element of  $C$ , true)
29:    $C \leftarrow \{v \in L : \sigma(v) = \sigma(b), (a, v) \in E(G)\}$ 
30:   if  $C = \emptyset$  then error ▷  $(G, \sigma)$  not sink-free
31:    $C^* \leftarrow \{v \in C : \exists w \in L \text{ with } \sigma(w) = \sigma(a), (v, w) \in E(G), (w, v) \in E(G)\}$ 
32:   if  $C^* \neq \emptyset$  then  $C \leftarrow C^*$ 
33:   return (random element of  $C$ , false)
34: end function

```



OBJECTIVE 1B

CHECK IF A MODIFIED VERSION OF THE BIC-CHERRY + EXPANSION PROCEDURE YIELDS NETWORKS THAT EXPLAIN WEAK BEST MATCH GRAPHS.



(OBJECTIVE „1C“)

FAILURE MODES VIA THE STRICT DEFINITION

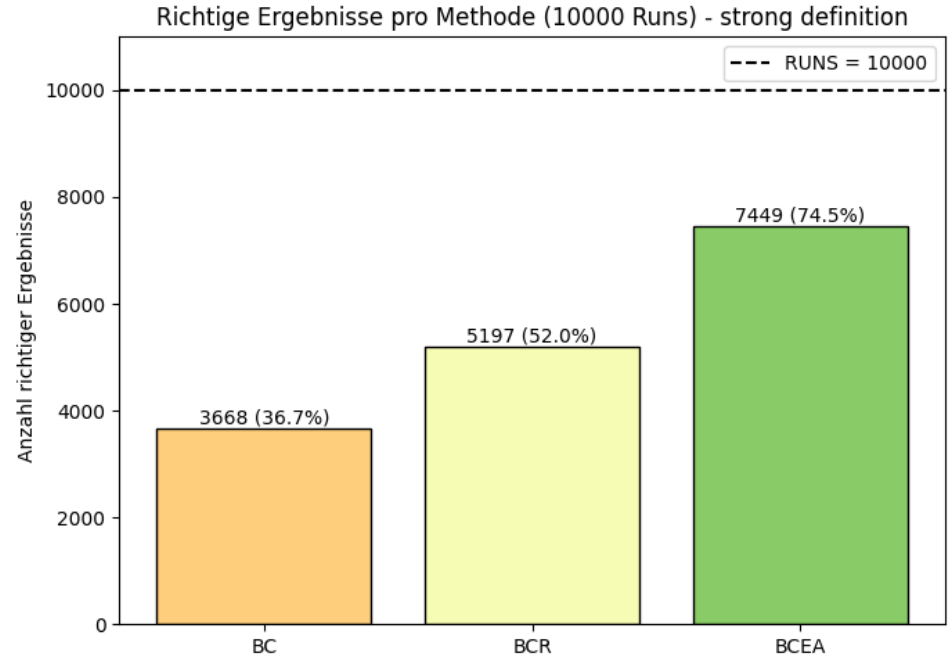
Algorithm 5: BIC-cherry + expand via safe-pairs (strict only)

Require: properly coloured digraph (G, σ) on L with the sccor-in-hub property

Ensure: leaf-coloured network (N, σ)

```
1:  $V(N) \leftarrow L \cup \{\rho\}$ ;  $E(N) \leftarrow \emptyset$ 
2: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do           ▷ BIC-cherry network
3:   add  $p_{xy}$  with edges  $(\rho, p_{xy}), (p_{xy}, x), (p_{xy}, y)$ 
4: end for
5: for all  $\{x, y\} \subseteq L$  with  $\sigma(x) \neq \sigma(y)$  do           ▷ expansion
6:   if  $(x, y) \notin E(G)$  and  $(y, x) \notin E(G)$  then       ▷ both directions missing
7:      $(x', y') \leftarrow \text{FINDJOINTCANDIDATES}(G, x, y)$ 
8:   else                                                 ▷ at most one direction missing
9:     for all  $(a, b) \in ((x, y), (y, x))$  with  $(a, b) \notin E(G)$  do  $b' \leftarrow$ 
       FINDCANDIDATE( $G, a, b$ )
10:    end if
11:    for all  $(a, b) \in ((x, y), (y, x))$  with  $(a, b) \notin E(G)$  do
12:      add new vertex  $q$  with edges  $(p_{xy}, q), (q, a), (q, b')$    ▷  $[ab : ab']$ 
13:      remove edge  $(p_{xy}, a)$                                        ▷ shortcut
14:    end for
15: end for
16: return  $(N, \sigma)$ 

23: function FINDJOINTCANDIDATES( $G, x, y$ )
24:    $X_1 \leftarrow \{v \in L : \sigma(v) = \sigma(x), (y, v) \in E(G)\}$ 
25:    $Y_1 \leftarrow \{v \in L : \sigma(v) = \sigma(y), (x, v) \in E(G)\}$ 
26:    $S \leftarrow \{(x', y') \in X_1 \times Y_1 : (x', y') \notin E(G), (y', x') \notin E(G)\}$ 
27:   if  $S \neq \emptyset$  then return random element of  $S$ 
28:    $X_2 \leftarrow \{v \in L \setminus \{x\} : \sigma(v) = \sigma(x)\}$ 
29:    $Y_2 \leftarrow \{v \in L \setminus \{y\} : \sigma(v) = \sigma(y)\}$ 
30:    $S \leftarrow \{(x', y') \in X_2 \times Y_2 : (x', y') \notin E(G), (y', x') \notin E(G)\}$ 
31:   if  $S \neq \emptyset$  then return random element of  $S$ 
32:   if  $X_1 \times Y_1 \neq \emptyset$  then return random element of  $X_1 \times Y_1$ 
33:   return random element of  $X_2 \times Y_2$            ▷ no safe pair exists
34: end function
```



OBJECTIVE 2

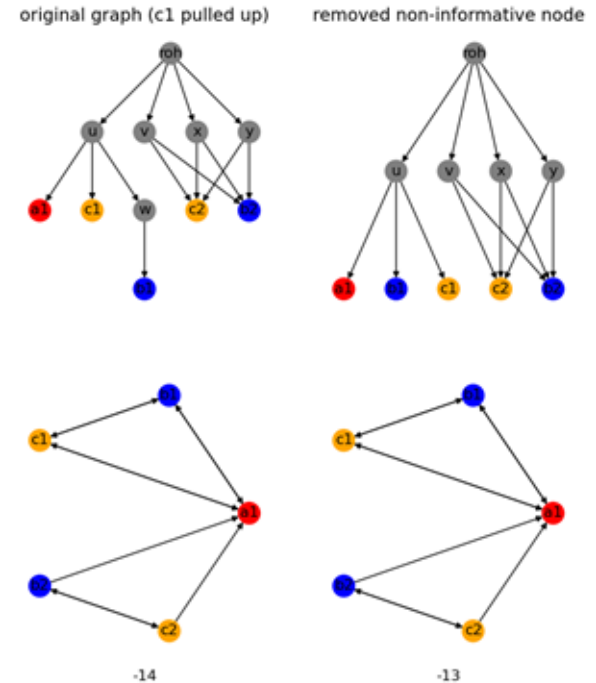
FIND A WAY TO EDIT THE EXPLAINING NETWORKS FOR (WEAK) BEST MATCHES TO BECOME MORE TREE LIKE.

Allowed Editing Operations:

- Pulling up
- Pulling down
- Removing redundant nodes (nodes with the same parents and children)
- Removing non-informative nodes

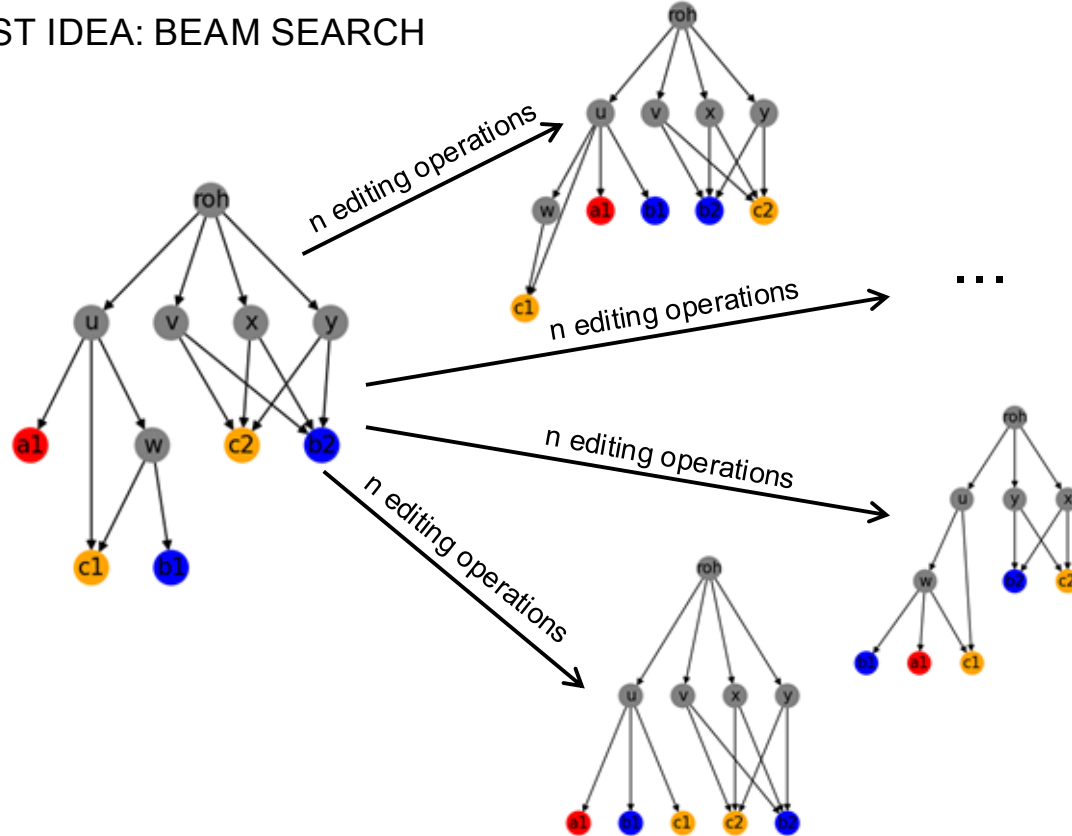
Tree-likeness measures:

- Negative Reticulation: $-(|E| - |V| + 1)$
- Negative Number of edges: $-|E|$



OBJECTIVE 2

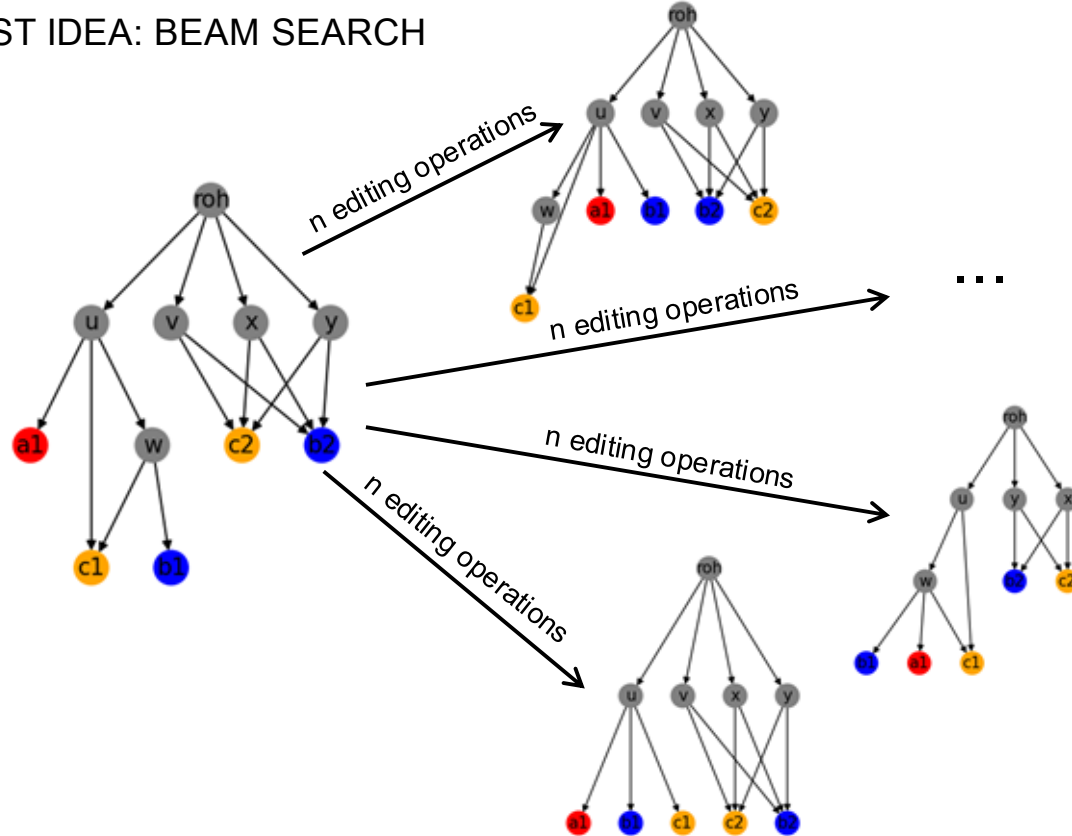
1ST IDEA: BEAM SEARCH



Select the best (in terms of tree likeness) m networks having the same BMG and repeat

OBJECTIVE 2

1ST IDEA: BEAM SEARCH



Select the best (in terms of tree likeness) m networks having the same BMG and repeat

Even with $n = m = 1$ on moderately sized graphs this took way too long

OBJECTIVE 2

FIND A WAY TO EDIT THE EXPLAINING NETWORKS FOR (WEAK) BEST MATCHES TO BECOME MORE TREE LIKE.

Algorithm 1 Greedily simplifying a network

Require: BIC cherry network N

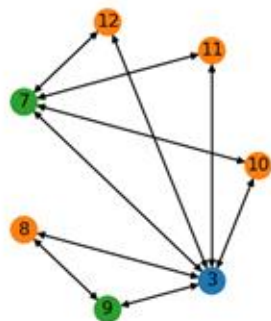
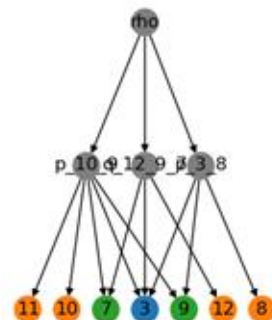
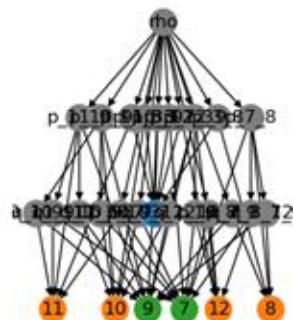
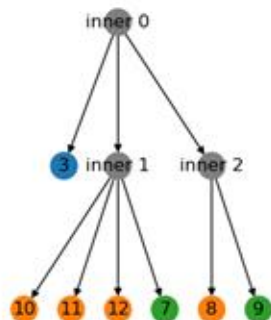
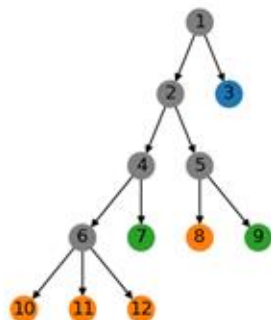
Ensure: Network N explaining the same best match graph with a higher tree likeness

```
1:  $bm_g \leftarrow \text{COMPUTE BMG}(N)$ 
2: repeat
3:    $improved \leftarrow \text{False}; k \leftarrow 0$ 
4:   Remove redundant nodes from  $N$ 
5:   Remove non-informative nodes from  $N$ 
6:    $score \leftarrow \text{TREELIKENESS}(N)$ 
7:   for  $(a, b, c) \in V^3$  with  $(a, b) \in E$  and  $(b, c) \in E$  do ▷ try pulling up
8:     Pull up  $(b, c)$  to  $a$ 
9:     if  $\text{TREELIKENESS}(N) > score$  and  $\text{COMPUTE BMG}(N) = bm_g$  then
10:      Accept pull-up operation
11:       $score \leftarrow \text{TREELIKENESS}(N)$ 
12:       $improved \leftarrow \text{True}$ 
13:    else
14:      Revert pull-up operation
15:    end if
16:  end for
```

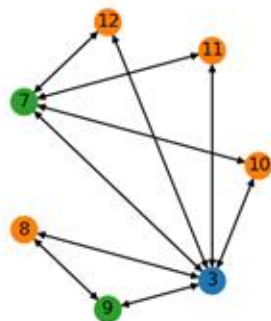
```
18:   ▷ try pulling down
19:   for  $(a, b, c) \in V^3$  with  $(a, b) \in E$  and  $(a, c) \in E$  and  $b \neq c$  do
20:     Pull down  $(a, b)$  to  $c$ 
21:     if  $\text{TREELIKENESS}(N) > score$  and  $\text{COMPUTE BMG}(N) = bm_g$  then
22:       Accept pull-down operation
23:        $score \leftarrow \text{TREELIKENESS}(N)$ 
24:        $improved \leftarrow \text{True}$ 
25:     else
26:       Revert pull-down operation
27:     end if
28:   end for
29:   if  $improved$  then
30:      $k \leftarrow 0$ 
31:   else ▷ plateau move
32:     Pick a random pull-up/pull-down Operation  $O$  with
33:      $\text{TREELIKENESS}(O(N)) = score$  and  $\text{COMPUTE BMG}(O(N)) = bm_g$ 
34:     if no such  $O$  exists then return  $N$ 
35:   end if
36:    $N \leftarrow O(N); k \leftarrow k + 1$ 
37: until  $k = 10$ 
38: return  $N$ 
```

OBJECTIVE 2

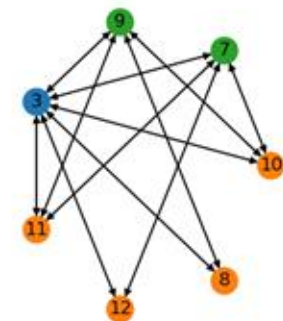
THINNESS



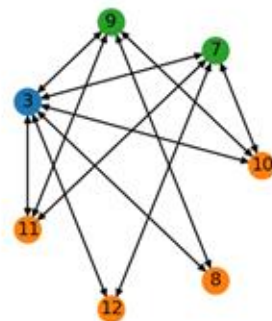
-11



-9



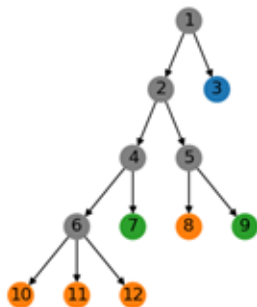
-66



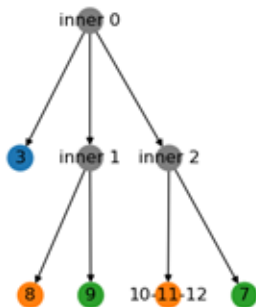
-14

OBJECTIVE 2

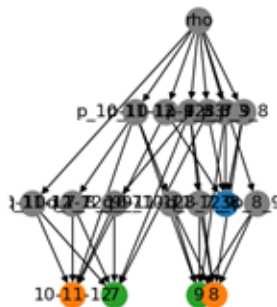
THINNESS



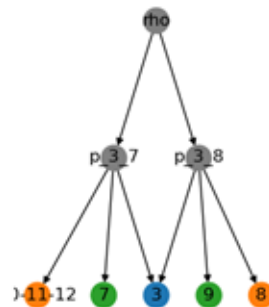
Original BMG:



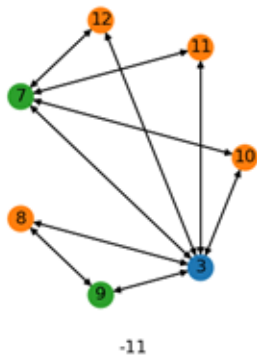
Thinness classes BMG:



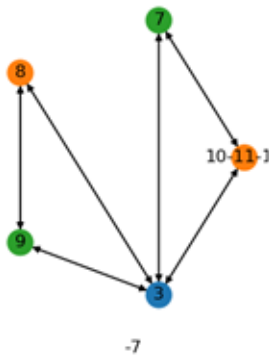
Thinness classes BMG:



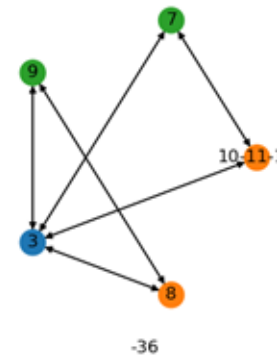
Thinness classes BMG:



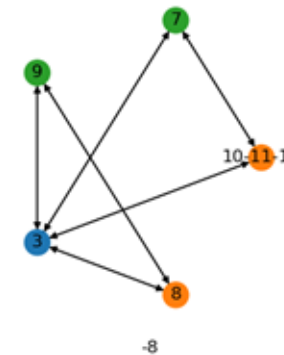
-11



-7



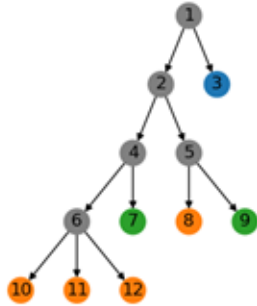
-36



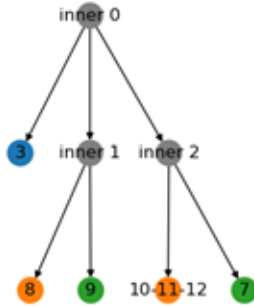
-8

OBJECTIVE 2

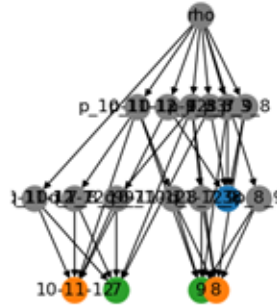
REMOVING DIAMONDS



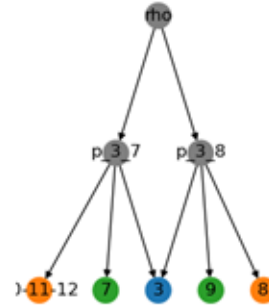
Original BMG:



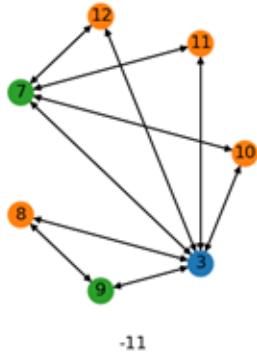
Thinness classes BMG:



Thinness classes BMG:



Thinness classes BMG:



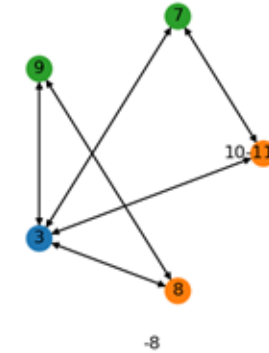
-11



-7



-36

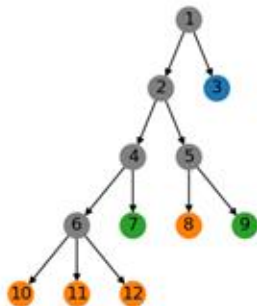


-8

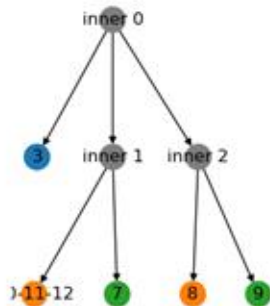
Algorithm 2 Remove diamond

- Pick a node n
- 2: if $\text{in-degree}(n) \geq 2$ then
 - Pick $p_1, p_2 \in \text{predecessors}(n)$
 - 4: Pick one $v \in \text{lca}(p_1, p_2)$
 - $E \leftarrow E \cup \{(v, n)\}$
 - 6: $E \leftarrow E \setminus \{(p_1, n), (p_2, n)\}$
 - end if

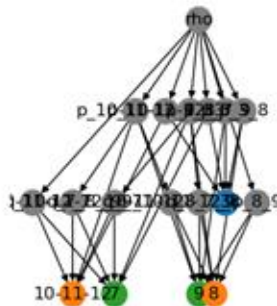
REMOVING DIAMONDS



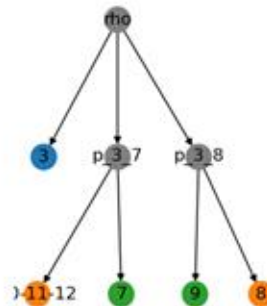
Original BMG:



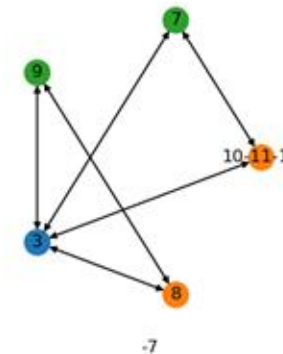
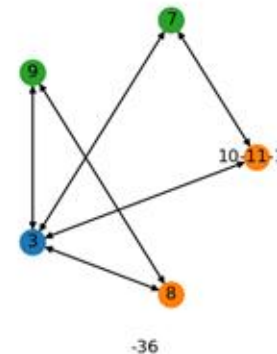
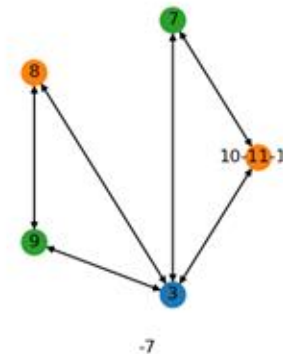
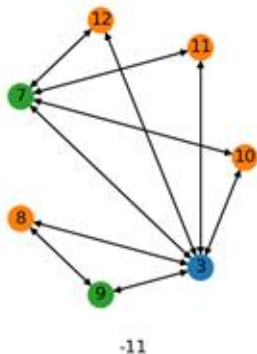
Thinness classes BMG:



Thinness classes BMG:



Thinness classes BMG:



Algorithm 2 Remove diamond

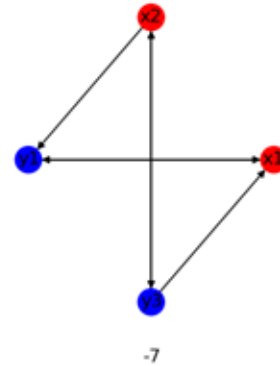
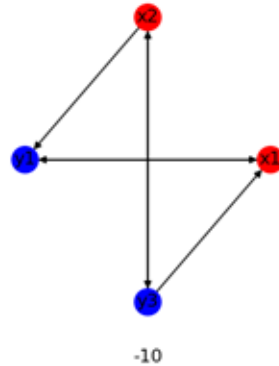
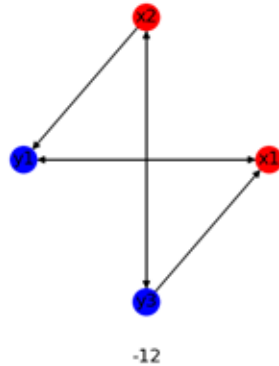
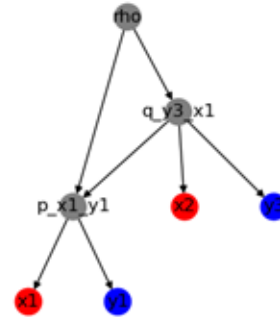
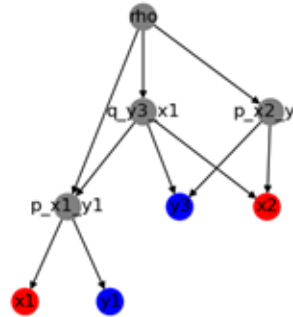
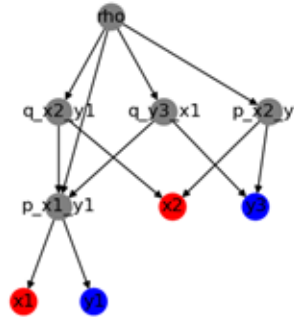
```

Pick a node n
2: if in-degree(n) ≥ 2 then
    Pick  $p_1, p_2 \in predecessors(n)$ 
4:   Pick one  $v \in lca(p_1, p_2)$ 
     $E \leftarrow E \cup \{(v, n)\}$ 
6:    $E \leftarrow E \setminus \{(p_1, n), (p_2, n)\}$ 
end if

```

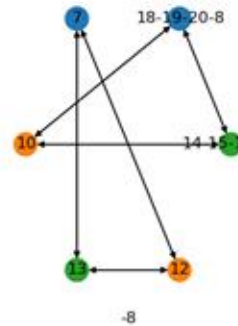
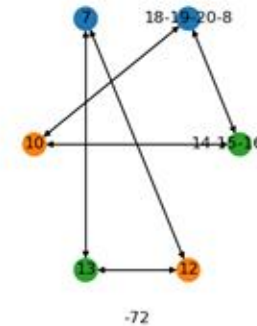
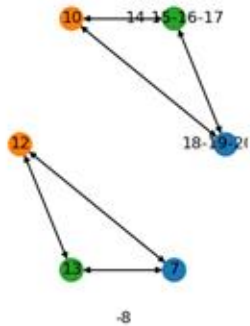
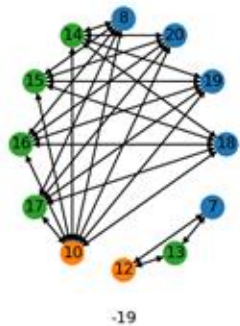
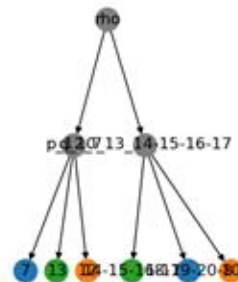
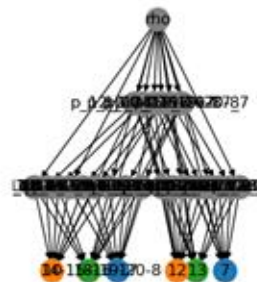
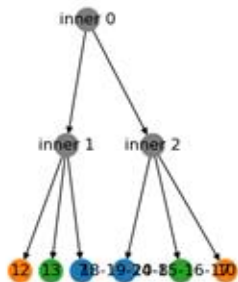
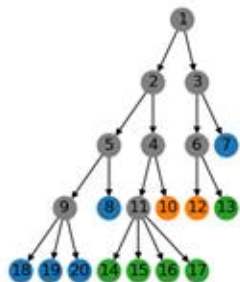
OBJECTIVE 2

COMBINING NODES



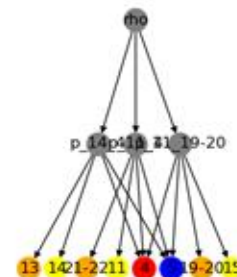
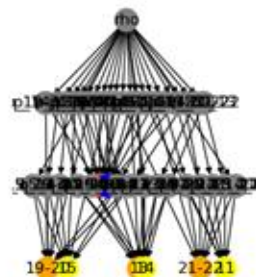
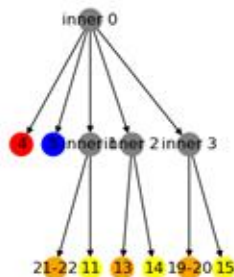
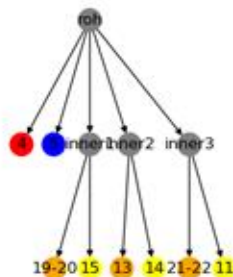
HEURISTIC WORKING

HEURISTIC WORKING

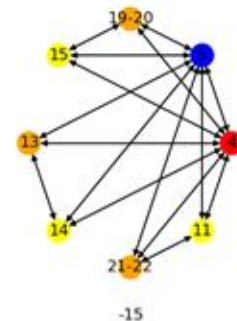
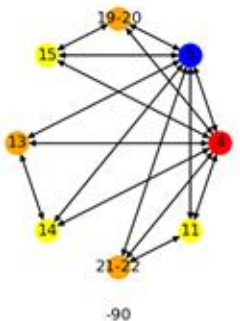
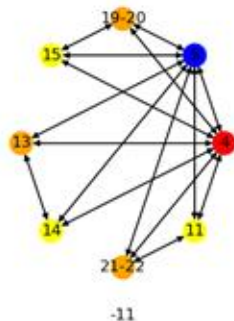
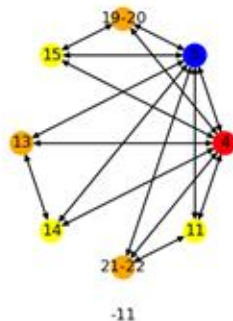


OBJECTIVE 2

HEURISTIC WORKING



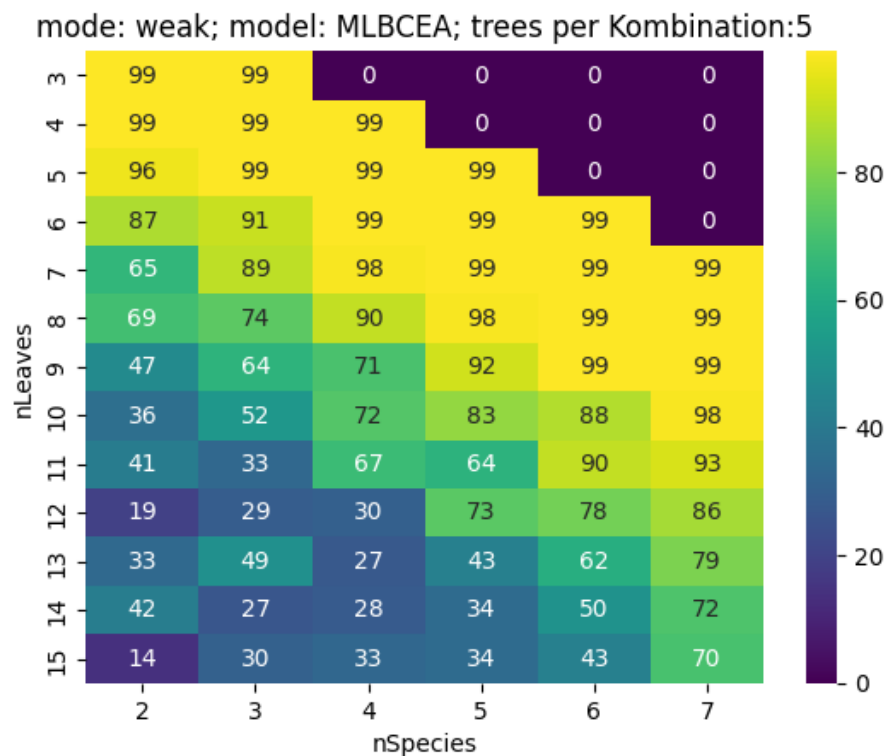
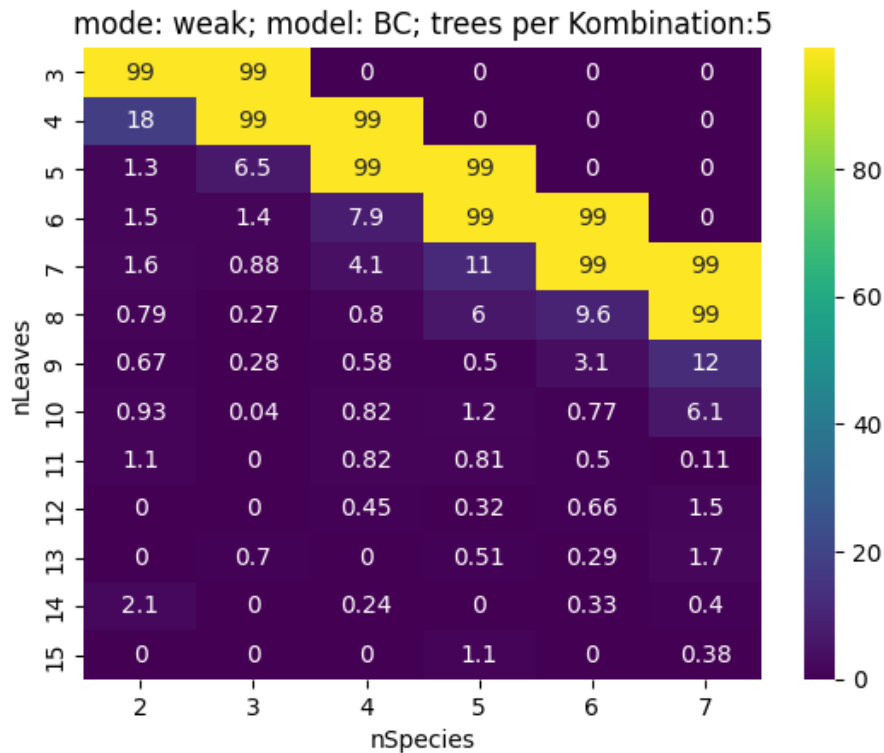
Solution: Solve
Diamonds for more than
two parents



More general: allow
more than one operation
at the same time

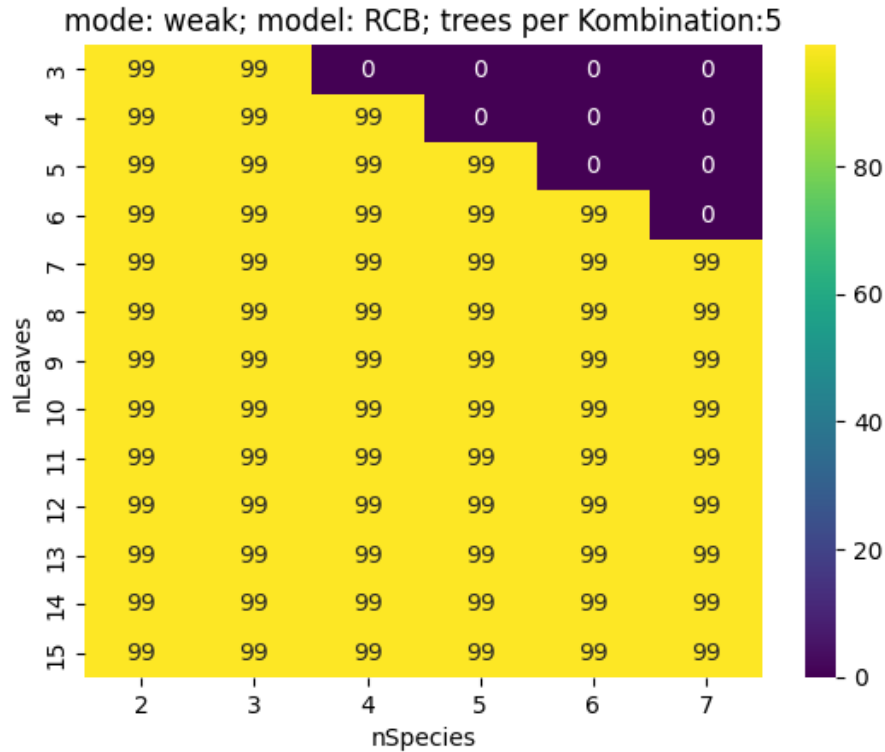
OBJECTIVE 3

RECOMMENDATION FOR FURTHER IMPROVEMENTS/MODIFICATIONS



OBJECTIVE 3

RECOMMENDATION FOR FURTHER IMPROVEMENTS/MODIFICATIONS



OBJECTIVE 3

RECOMMENDATION FOR FURTHER IMPROVEMENTS/MODIFICATIONS

OBJECTIVE 3

REVERSE BIC CHERRY EXPLAINED

Algorithm 7: Reverse BIC-cherry

Require: properly coloured digraph (G, σ) on L with the sicer-in-hub property, root ρ

Ensure: leaf-coloured network (N, σ) ; $c(a, b)$: node(s) of N that make b a best match of a

1: $V(N) \leftarrow L \cup \{\rho\}$; $E(N) \leftarrow \emptyset$; $W \leftarrow \emptyset$

2: if $|L| = 2$ then add edges (ρ, x) for all $x \in L$ and return (N, σ)

3: for all $\{x, y\} \subseteq L$ with $(x, y), (y, x) \in E(G)$ do

▷ step 1: reciprocal edges

4: add ρ_{xy} with edges $(\rho, \rho_{xy}), (\rho_{xy}, x), (\rho_{xy}, y)$;

5: end for

OBJECTIVE 3

REVERSE BIC CHERRY EXPLAINED

Algorithm 7: Reverse BIC-cherry

Require: properly coloured digraph (G, σ) on L with the sicc-in-hub property, root ρ

Ensure: leaf-coloured network (N, σ) ; $c(a, b)$: node(s) of N that make b a best match of a

1: $V(N) \leftarrow L \cup \{\rho\}$; $E(N) \leftarrow \emptyset$; $W \leftarrow \emptyset$

2: if $|L| = 2$ then add edges (ρ, x) for all $x \in L$ and return (N, σ)

3: for all $\{x, y\} \subseteq L$ with $(x, y), (y, x) \in E(G)$ do ▷ step 1: reciprocal edges

4: add ρ_{xy} with edges $(\rho, \rho_{xy}), (\rho_{xy}, x), (\rho_{xy}, y)$:

5: end for

6: for all $(x, y) \in E(G)$ with $(y, x) \notin E(G)$ do ▷ step 2: partner of x 's colour

7: $Z \leftarrow \{z \in L : \sigma(z) = \sigma(x), (y, z), (z, y) \in E(G)\}$

8: for all $z \in Z$ do add q with edges $(\rho, q), (q, x), (q, \rho_z)$

9: if $Z = \emptyset$ then $W \leftarrow W \cup \{(x, y)\}$ ▷ left for step 3

10: end for

11: while $W \neq \emptyset$ do ▷ step 3: search from y

12: for all $(x, y) \in W$ whose required $c(y, \cdot)$ exist do

13: for all $t \in \text{FINDTARGETS}(G, x, y)$ do add r with edges $(\rho, r), (r, x), (r, t)$:

14: $W \leftarrow W \setminus \{(x, y)\}$

15: end for

16: end while

17: return (N, σ)

OBJECTIVE 3

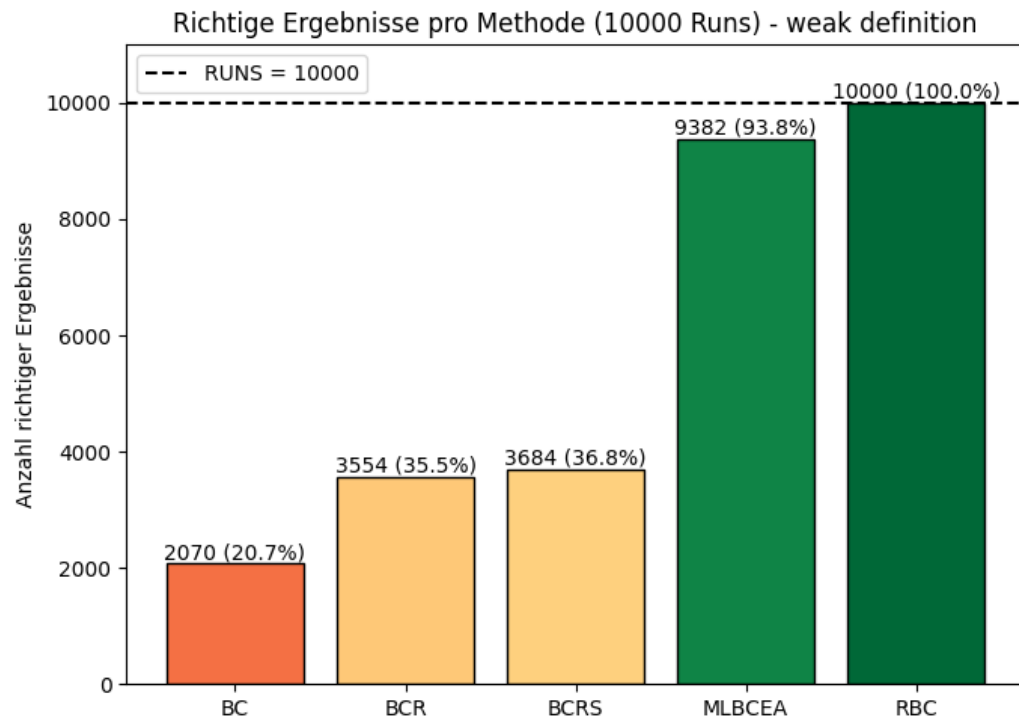
REVERSE BIC CHERRY EXPLAINED

Function FindTargets (step 3 of Algorithm 7)

```
1: function FINDTARGETS( $G, x, y$ )  
2:   for  $d = 1, 2, \dots$  do ▷ outward from  $y$ , nearest first  
3:      $H_d \leftarrow \{w \in L \setminus \{x\} : \sigma(w) = \sigma(x), \text{dist}_G(y, w) = d\}$  ▷ nodes of  $x$ 's colour  
4:      $T_d \leftarrow \{c(y, u) : y \rightarrow u \rightarrow \dots \rightarrow w \text{ is a shortest path to some } w \in H_d\}$   
5:      $T_d^* \leftarrow \{t \in T_d : (x, \ell) \in E(G) \text{ for all } \ell \in L(t) \text{ with } \sigma(\ell) \neq \sigma(x)\}$  ▷ clean cluster  
6:     if  $T_d^* \neq \emptyset$  then return  $T_d^*$   
7:   end for  
8:   return  $T_{d_0}$  for the smallest  $d_0$  with  $H_{d_0} \neq \emptyset$  ▷ fallback: nearest  
9: end function
```

OBJECTIVE 3

REVERSE BIC CHERRY: PERFORMANCE



OBJECTIVE 3

REVERSE BIC CHERRY SIMPLIFIED EXPLAINED

Algorithm 8: Reverse BIC-cherry, simplified

Require: properly coloured digraph (G, σ) on L with the sior-in-hub property

Ensure: leaf-coloured network (N, σ)

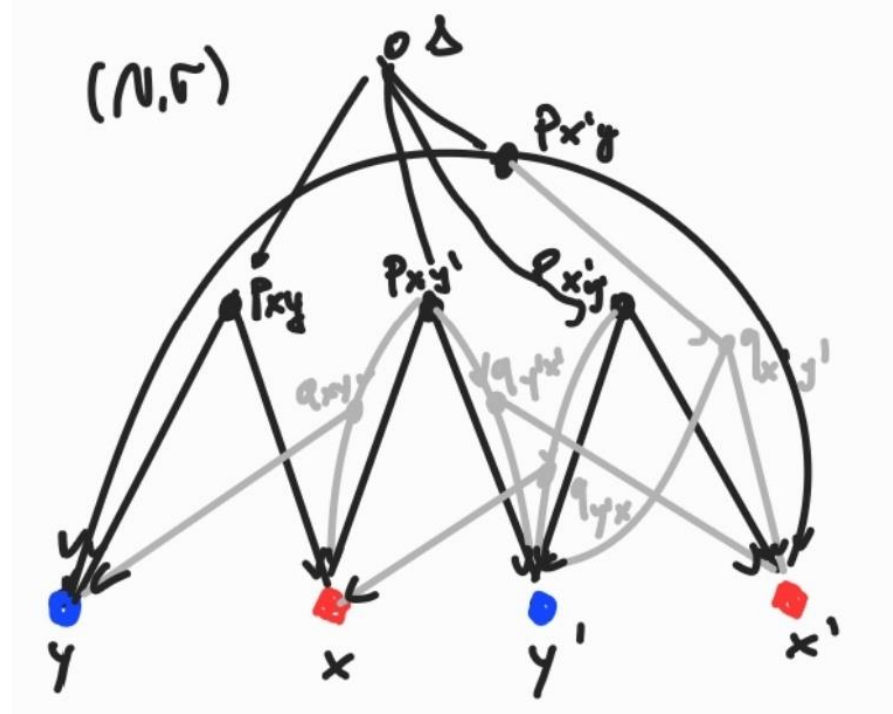
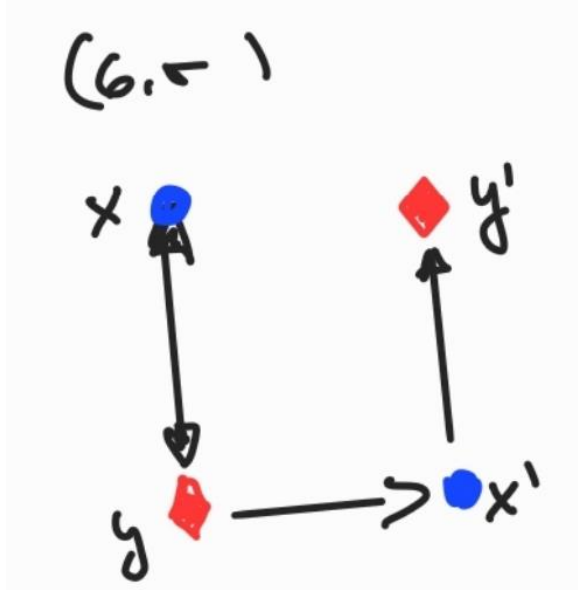
```
1:  $k \leftarrow 0$ ;  $V(N) \leftarrow \{\rho_0\}$ ;  $E(N) \leftarrow \emptyset$ 
2: repeat
3:    $A \leftarrow \{v \in V(G) : N^+(v) = N^-(v) = V(G) \setminus \{v\}\}$ ;  $\text{ATTACH}(A)$ 
4:    $B \leftarrow \{v \in V(G) : N^-(v) = \emptyset, N^+(v) = \{u : \sigma(u) \neq \sigma(v)\}\}$ ;  $\text{ATTACH}(B)$ 
5: until  $A = B = \emptyset$ 
6: if  $E(G) = \emptyset$  then
7:   add edges  $(\rho_k, v)$  for all  $v \in V(G)$ ; remove  $\rho_k$  if it has no children
8: else
9:   add Algorithm 7 on the remaining  $(G, \sigma)$  with root  $\rho_k$  to  $N$ 
10: end if
11: return  $(N, \sigma)$ 

12: procedure  $\text{ATTACH}(S)$ 
13:   if  $S \neq \emptyset$  then
14:     add edges  $(\rho_k, v)$  for all  $v \in S$ ; remove  $S$  from  $G$ 
15:     add  $\rho_{k+1}$  with edge  $(\rho_k, \rho_{k+1})$ ;  $k \leftarrow k + 1$ 
16:   end if
17: end procedure
```

▷ peel off leaves from the top
▷ reciprocal to all
▷ no incoming

OBJECTIVE 3

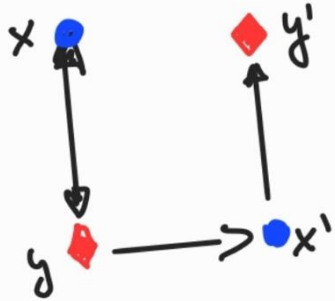
NON-BMGS CAN NOT BE USED TO GENERATE THE CORRECT NETWORKS



OBJECTIVE 3

NON-BMGS CAN NOT BE USED TO GENERATE THE CORRECT NETWORKS

(6.5)



OBJECTIVE 3

PERFORMANCE CHECK ON TREES

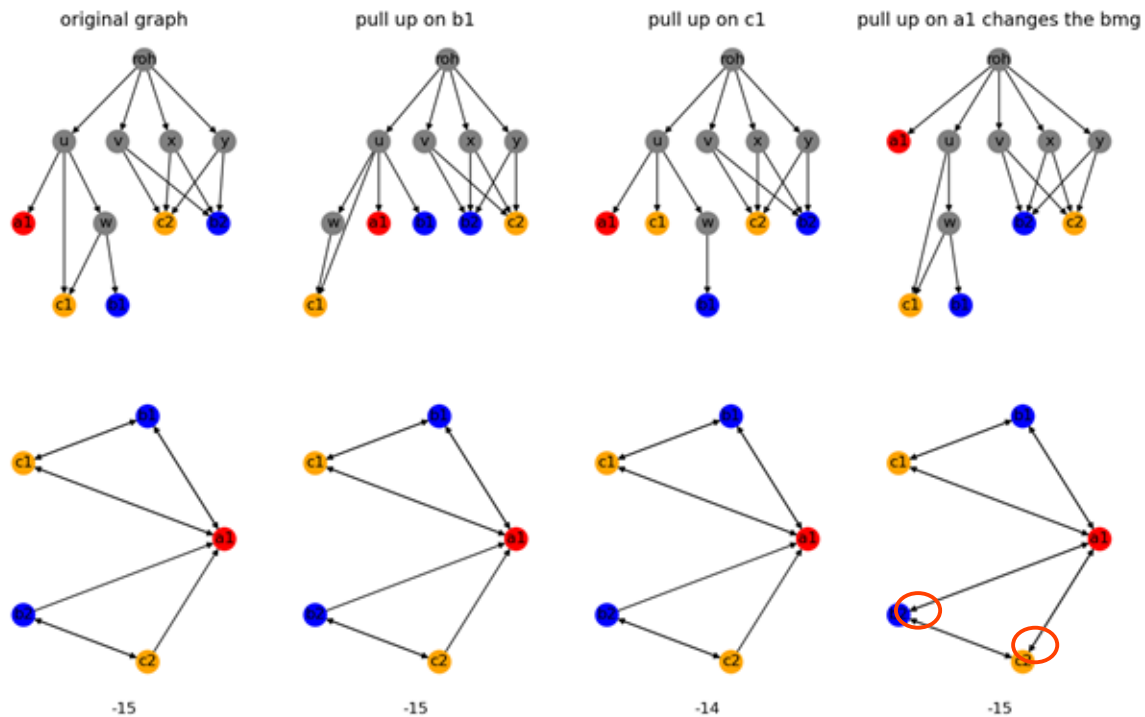
OBJECTIVE 3

PERFORMANCE CHECK ON NETWORKS

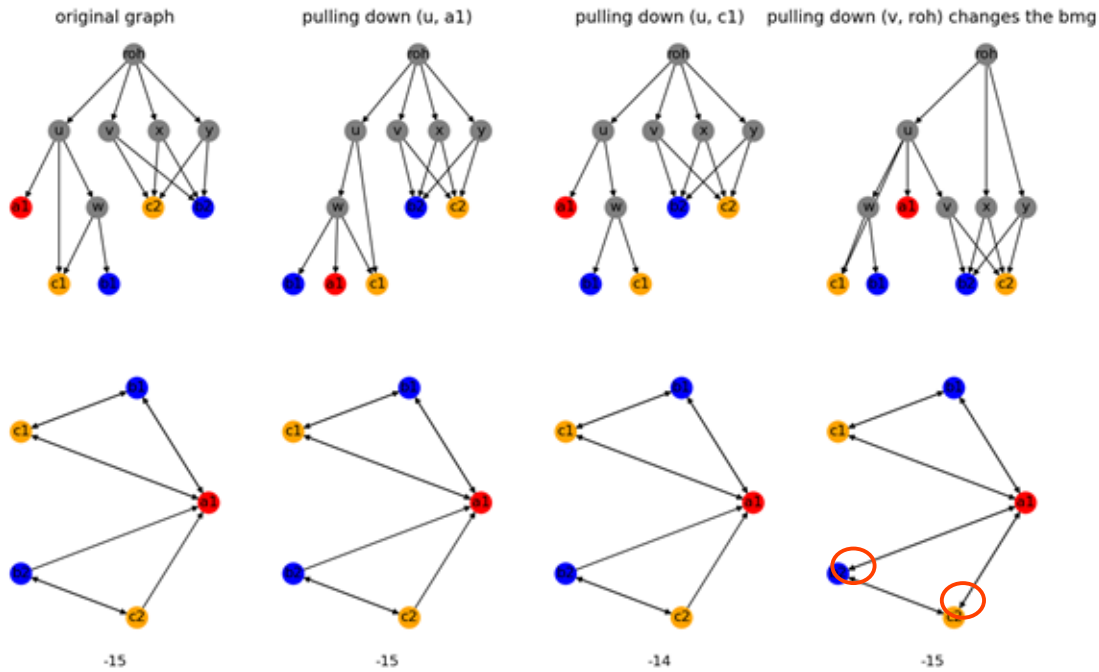
BIEREN DANK!
(UND PROST)



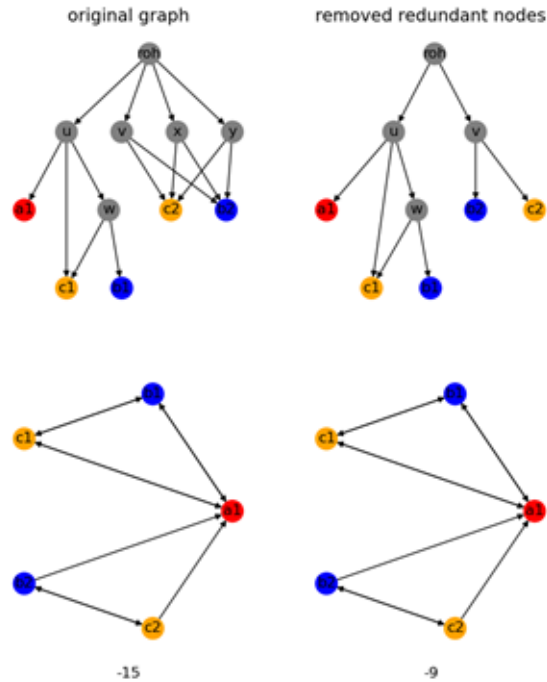
PULLING UP



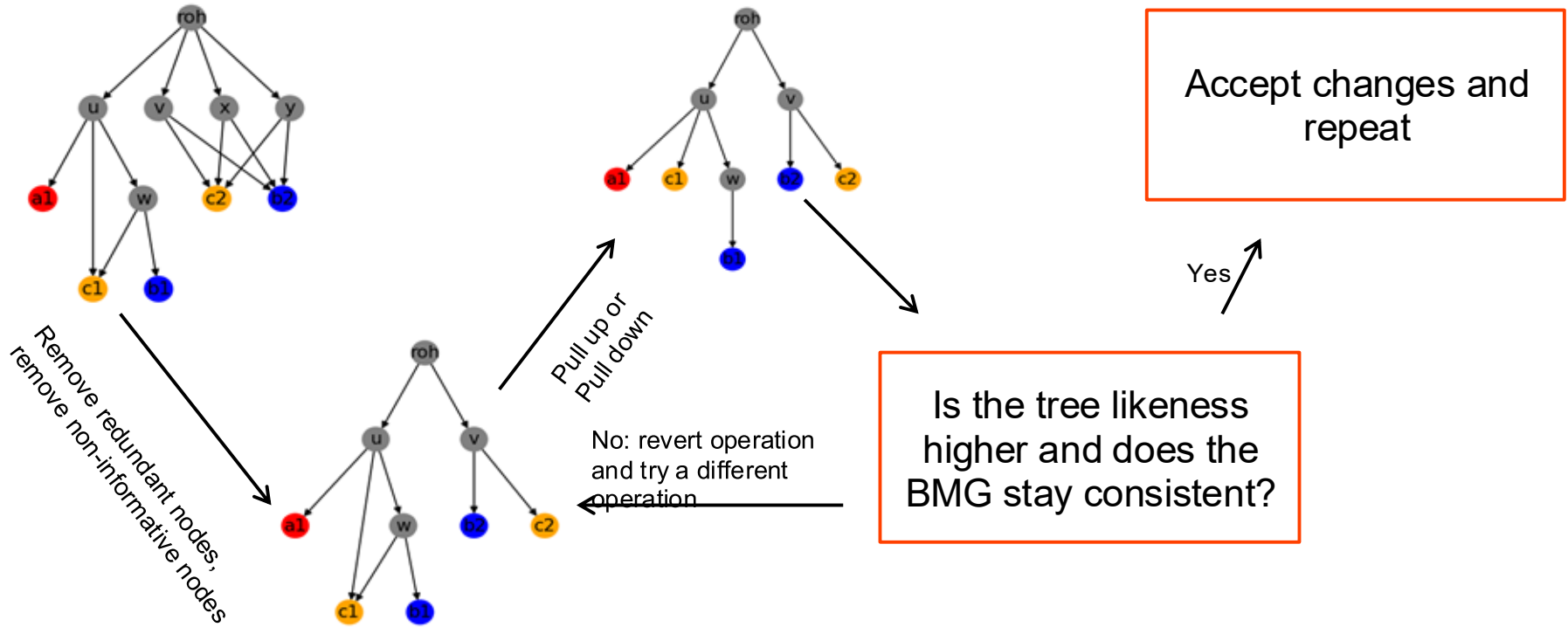
PULLING DOWN



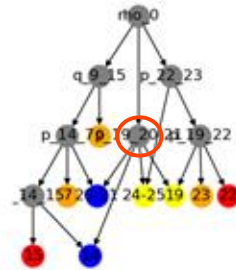
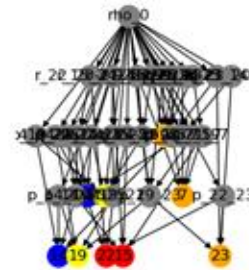
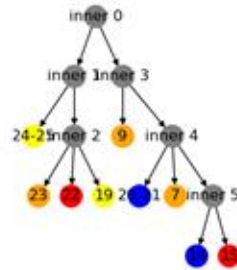
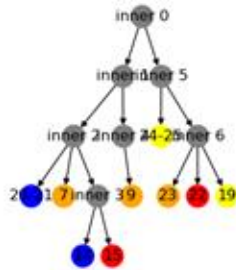
REMOVE REDUNDANT NODES



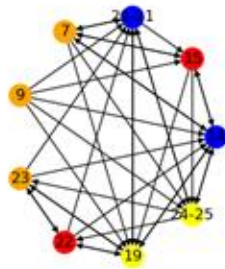
SOLUTION: GREEDY SEARCH



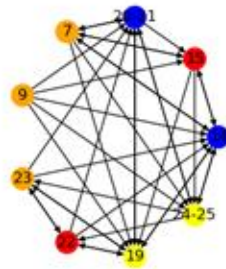
HEURISTIC NOT WORKING



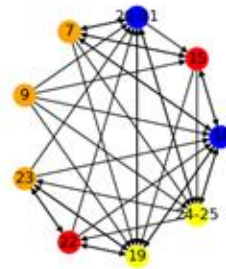
Solution: Remove nodes



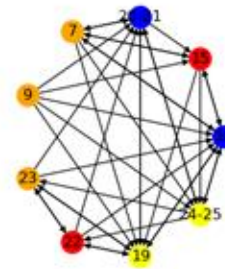
-15



-14



-81



-19